APPENDIX D.7

Implemented Security and Privacy Measures for CurriculumAI

This appendix documents security and privacy safeguards at CurriculumAI's account, browser, ingestion, database and external AI interfaces. It expands the design overview in Section 4.10 and the implementation account in Chapter 5. The described controls mitigate specified risks; their presence does not establish comprehensive security assurance.

Account access and session protection

Django authenticates users by email and stores salted password hashes. Password validators apply configured strength requirements. Django REST Framework authenticates browser API requests through sessions. Session cookies are HttpOnly, and session and CSRF cookies use SameSite=Lax. The React API client obtains a fresh CSRF token before state-changing requests. Login, registration, password recovery and guest question submission explicitly apply CSRF checks. Configured host and origin allowlists restrict accepted hosts and cross-origin access, with credentialed CORS limited to specified API origins.

Registration uses hashed six-digit email verification codes with a ten-minute expiry, sixty-second resend cooldown and five-attempt limit. Completion requires a verified grant associated with the browser session. Password recovery uses expiring Django reset tokens, generic acknowledgements and a mailbox cooldown keyed using a hash-based message authentication code (HMAC). Sensitive account changes require current-password confirmation; replacement email addresses undergo verification. Request throttles limit repeated authentication, verification and question-answering attempts.

Implementation references: backend/config/settings.py; backend/accounts/; backend/qa/views.py

Access to private records and retention

Server-side permissions restrict source administration and model configuration to authorised administrators. Student conversation queries are scoped to the authenticated owner before decryption; staff accounts cannot access the student conversation APIs. Administrator reporting provides aggregate source-usage counts and an Active/Inactive indicator based on a 90-day window, rather than student conversation contents or exact activity timestamps.

The Python cryptography package uses Fernet to encrypt stored conversation titles and message contents. A separate backend encryption key is supplied through runtime configuration outside PostgreSQL. The authenticated encrypted payload includes the expected record and field context so that misplaced ciphertext can be rejected. Chat APIs use non-cacheable responses. Guest turns remain in browser memory and are not saved to CurriculumAI's conversation database; content sent to the external AI provider is subject to that provider's data-handling arrangements.

Conversation deletion removes dependent messages and citations. Student application account deletion provides a short undo period before cleanup; this is distinct from the research data retention arrangements described in Section 3.8. Historical citation snapshots preserve source titles and locations when course sources are deleted.

Implementation references: backend/chat/encryption.py; backend/chat/selectors.py; backend/chat/models.py; backend/accounts/student_account.py; backend/knowledge/usage.py

Document uploads and website ingestion

Only authorised administrators can register course sources. Uploaded PDF, PPTX and DOCX files are checked against supported formats and their signatures or Office package structure. A streaming upload handler enforces a limit of 200,000,000 bytes (200 MB) per file. Display filenames are sanitised, while UUID-based storage names are generated independently of user-supplied paths. The API and ingestion worker access files through private media storage; access-controlled endpoints expose permitted source and citation details without a public download route to the original uploads.

Extraction uses pypdf, python-pptx and python-docx subject to limits on document units and extracted text. Office packages are checked against expanded-size limits before parsing. SHA-256 content hashes and PostgreSQL uniqueness constraints reject duplicate binary registrations, including concurrent submissions. The ingestion worker marks a source INDEXED only after passage, metadata and vector checks succeed; retrieval considers only eligible indexed material.

Website ingestion uses urllib3 with public-destination checks to mitigate server-side request forgery (SSRF). The fetcher validates requested destinations, DNS answers and redirect targets, and connects to a checked IP address while preserving HTTPS hostname and certificate checks. Redirect counts, response size and fetching duration are bounded. Fetch requests omit cookies and refuse compressed responses. BeautifulSoup extracts page text for ingestion.

Implementation references: backend/knowledge/validation.py; backend/knowledge/upload_handlers.py; backend/knowledge/extraction/common.py; backend/knowledge/extraction/web.py

Data transmitted to OpenAI

The backend retains the OpenAI API key and calls provider APIs through server-side adapters. Embedding requests contain extracted course-passage text or a retrieval query; a follow-up query may include bounded earlier student questions. Generation requests contain the current question and bounded conversation context. Tier 1 additionally receives selected course passages. For registered students, recent history is loaded only after ownership checks and is subject to message and character limits.

Constructed requests omit account identity fields, but user-entered questions, conversation context and source documents may themselves contain personal information. The formal evaluation used predefined, non-personal Scala questions as described in Section 3.8. Accordingly, private storage within CurriculumAI does not mean that submitted text is never processed by an external provider.

The generation adapter sets **store=False** for the OpenAI Responses API, disabling storage of that generated response for later API retrieval. This setting does not by itself exclude separate provider-side retention under applicable abuse-monitoring policies. It is a generation-request setting and should not be interpreted as a claim that all embedding or generation data is retained for zero days.

Implementation references: backend/knowledge/embeddings/openai_provider.py; backend/qa/context.py; backend/qa/generation/openai_provider.py

Input handling and response integrity

The OpenAI Responses API receives fixed backend instructions separately from structured student input. Questions, recent history and retrieved passages are treated as untrusted data. Tier 1 instructions use current retrieved passages as course evidence and history as context for interpreting the question. Tier 2 receives the current question and bounded history for external Scala assistance, excluding the current retrieved passages and any rejected Tier 1 output. Neither generation request supplies browsing, code-execution or other callable tools.

The adapter requests strict JSON Schema output, after which Django independently validates returned content. For Tier 1, regular expressions and set comparisons check that inline citation markers correspond to unique citation entries. Each quotation must be an exact contiguous substring of its associated retrieved passage. A non-empty quotation

mismatch permits at most one regeneration under repeated validation. Source eligibility is rechecked before acceptance. For registered students, transaction.atomic() groups the encrypted assistant answer and citation writes so failures cannot leave a partially saved answer.

Tier 2 validation rejects course-citation markers and numerical footnote references. Django adds the external-origin label and reason-specific notice on the server. These layered controls constrain accepted output and provide structural and verbatim citation traceability. They do not establish that every generated claim is semantically supported by its cited passage; this is assessed separately in Chapter 6.

Implementation references: backend/qa/generation/; backend/qa/fallback/; backend/qa/generation/openai_provider.py

Browser rendering, diagnostics and deployment

React renders student-submitted questions as plain text. Assistant Markdown is rendered with raw HTML disabled; external images are replaced by alternative text, and clickable links are limited to HTTP or HTTPS with opener and referrer protections. The backend returns sanitised API errors. Request-local QA diagnostics record processing stages, error or outcome codes and counts without logging prompts, conversation content or account identifiers in those diagnostic records.

The supplied local Docker Compose configuration binds frontend and backend ports to loopback and keeps PostgreSQL on the internal container network. The frontend and backend services use no-new-privileges, and the Django container runs as a non-root user. Git and Docker ignore rules exclude specified local secret and private-media paths from tracked source or build inputs as configured; any separately distributed handover package must be checked independently.

HTTPS redirects, secure-cookie settings and HTTP Strict Transport Security depend on deployment configuration and are not claimed to be active in every local environment. The safeguards documented here mitigate selected risks but are not a substitute for a comprehensive penetration test, independent security assessment or production deployment review. Chapter 7.5 records the corresponding assurance limitations.

Implementation references: frontend/src/components/chat/MessageList.jsx; backend/config/middleware.py; backend/qa/diagnostics.py; compose.yaml; Dockerfile; .gitignore; .dockerignore